

Test et qualité

Yohan Boichut

Version 2.0

Sommaire

Who am i	1
Organisation	1
Le cours	1
1. Introduction	2
1.1. Fail 1	2
1.2. Fail 2	2
1.3. Fail 3	2
1.4. Fail 4	2
1.5. Fail 5	2
1.6. Fail 6 (1/2)	3
1.7. Fail 6 (2/2)	3
1.8. Fail 7 (1/5)	4
1.9. Fail 7 (2/5)	4
1.10. Fail 7 (3/5)	5
1.11. Fail 7 (4/5)	5
1.12. Fail 7 (5/5)	6
1.13. Quand Konami s'y met aussi	6
1.14. Quand Konami s'y met aussi	7
1.15. Quand Konami s'y met aussi	7
1.16. Quand Konami s'y met aussi	8
1.17. Le test... une activité simple ?	8
1.18. Motivations du test	9
1.19. Qualité du logiciel (1/2)	9
1.20. Qualité du logiciel (2/2)	9
1.21. Le test, what is it?	9
1.22. Test dynamique	9
1.23. Test dynamique : deux méthodes	10
1.24. Types de tests	10
1.25. Test dans le cycle de vie du logiciel	10
1.26. Quelques références	10
2. JUnit	12
2.1. Tests unitaires	12
2.2. Vocabulaire JUnit	12
2.3. Principes élémentaires	12
2.4. JUnit - programmation d'une classe de test	12
2.5. Assertions	13
2.6. Ce qu'il faut retenir : cas de test (1/3)	13
2.7. Ce qu'il faut retenir : fixtures (2/3)	13

2.8. Ce qu'il faut retenir : test-to-fail (3/3)	14
2.9. Mise en application sur l'écurie	14
2.10. Junit 5 — Jupiter	14
2.11. Assertions plus précises	14
2.12. Quelques annotations un peu différentes	15
3. Tests fonctionnels	16
3.1. A tous les niveaux	16
3.2. Quality center (1/2)	16
3.3. Quality center (2/2)	17
3.4. Test fonctionnel	18
3.5. Approche du test fonctionnel	18
3.6. Les problématiques liées au test fonctionnel	19
3.7. Partitions d'équivalence	19
3.8. Exemple de partitions d'équivalence	19
3.9. Combinaison des partitions d'équivalence	20
3.10. Aux limites il se passe toujours des choses bizarres	20
3.11. Limiter le nombre de combinaisons	20
3.12. Approche PairWise	20
3.13. Exemple de l'approche PairWise	20
3.14. Tests fonctionnels à partir d'interfaces	21
3.15. Approche fonctionnelle pour le test de classes	21
3.16. Réseau social (1/4)	21
3.17. Réseau social : raisonnement (2/4)	21
3.18. Réseau social : implémentation des cas de tests (3/4)	22
3.19. Les astuces de sioux	22
3.20. Application pour les assertions du cas de test	22
3.21. Réseau social : Bilan (4/4)	23
3.22. Test fonctionnel → Test d'intégration	23
3.23. Classe de test abstraite	23
3.24. Classe de test abstraite	24
3.25. Classe de test fille	24
3.26. Classe de test fille	24
3.27. Un peu de recul sur la méthodologie	25
3.28. Compétences à acquérir	25
4. Tests fonctionnels — Applications Web	26
4.1. Pour une IHM donnée, ...	26
4.2. Selenium	26
4.3. Selenium vs Paris en ligne	26
4.4. Selenium et automatisation	26
4.5. Un petit point d'histoire en premier lieu	27
4.6. Selenium Grid	27

4.7. Un exemple pratique ?	27
5. Tests structurels	28
5.1. Principes fondamentaux	28
5.2. Critères de couverture	28
5.3. Impact au niveau des jeux de données	28
5.4. Toutes les instructions	28
5.5. Tous-les-arcs	29
5.6. Et le bug ?	29
5.7. SonarQube via Docker (1/2)	30
5.8. Configuration SonarQube	30
5.9. Dans votre projet Maven 1/	30
5.10. Dans votre projet Maven 2/	31
5.11. Dans votre projet Maven 2/	31
5.12. Conclusion	31
6. Les Mocks	33
6.1. Concept	33
6.2. Motivations	33
6.3. Exemple de cas intéressant	33
6.4. Processus	33
6.5. Ce qu'on ne veut pas voir	33
6.6. Ce qu'on attend	34
6.7. On teste des Mocks ?	34
6.8. Etapes indispensables pour utiliser des Mocks	34
6.9. Mocking partiel	35
6.10. Mocking partiel cas extrême	35
6.11. Exemple de cas intéressant	35
7. Mockito	36
7.1. Introduction	36
7.2. Installation de Mockito	36
7.3. Installation de Mockito avec Junit5	36
7.4. Création de mocks	36
7.5. Configuration de comportements de mocks	37
7.6. Vérification de comportements de mocks	37
7.7. Mocking partiel avec Mockito	37
7.8. Test de la classe précédente	37
7.9. Attention avec les annotations	38
7.10. Alternative	38
7.11. Là où ça devient obscure....	39
7.12. Le côté obscure de Spy 1/	39
7.13. Le côté obscure de Spy 2/	39
7.14. Le côté obscure de Spy 3/	39

7.15. Eclairages du côté obscur de Spy et Mock	40
7.16. Mocking — HOW TO avec méthode	40
7.17. En pratique	40
7.18. Spécifier une levée d'exception	40
7.19. Petit cas d'étude	41
7.20. Classe de test bien faite	41
7.21. Cas complexes	43
7.22. Les ArgumentMatchers (1/3)	43
7.23. Les ArgumentMatchers (2/3)	43
7.24. Les ArgumentMatchers (3/3)	43
8. Tests structurels en Struts 2	44
8.1. Rappel de la machine Struts 2	44
8.2. Paris en ligne - action AjouterMatch (1/2)	44
8.3. Paris en ligne - action AjouterMatch (2/2)	44
8.4. Action AjouterMatch (1/2)	45
8.5. Action AjouterMatch (2/2)	45
8.6. Coup d'oeil sur le struts.xml	45
8.7. Comment tester une action ?	46
8.8. Road to test AjouterPari action	46
8.9. Road to test AjouterPari action	46
8.10. Etapes pour un cas de test	47
8.11. Exemple sur testOk()	47
8.12. Exemple sur testOk()	47
8.13. Exemple sur testOk()	48
8.14. Exemple sur testOk()	48
8.15. Exemple sur testOk()	48
8.16. Tuto YouTube	49
9. Comment tester une API REST ?	50
9.1. Concept	50
9.2. Des éléments de réponse	50
9.3. Exemple de service REST	50
9.4. Méthodologie	50
9.5. Identification	51
9.6. Préparation	51
9.7. Exécution	51
9.8. Mise en application - Classe de test	51
9.9. Mise en application	52
9.10. Mise en application	52
9.11. Quelques informations	52
10. Tester une API REST SpringBoot 3 : JWT	53
10.1. Contexte	53

10.2. Etude de cas	53
10.3. Politique de sécurité de l'application	53
10.4. /api/register	53
10.5. Les champs de ma classe de test	54
10.6. Mise en application : inscription	54
10.7. Mise en application : inscription /2	55
10.8. /api/admin avec rôle ADMIN	55
10.9. Ecriture du test avec token	56
10.10. Possibilité de le faire sans Token généré	56
11. Comment tester une application Spring MVC ?	57
11.1. Principes	57
11.2. Exemple de contrôleur Spring MVC — EvenementControleur.java	57
11.3. Test structurel	58
11.4. Identification	58
11.5. Structure de la classe de Test	58
11.6. Premier cas de test - erreurs en input	58
11.7. Premier cas de test - erreurs en input	59
11.8. Définition du cas de test	59
11.9. Deuxième cas de test - Tout va bien	59
11.10. Deuxième cas de test - Tout va bien	60
11.11. Définition du cas de test	60
11.12. Troisième cas de test — UtilisateurInconnuException	61
11.13. Définition du cas de test	61
11.14. Outils indispensables	61

Who am i

📧 Yohan Boichut

[address card o] yohan.boichut@univ-orleans.fr

[twitter] @YohanBoichut

[github] yohanboichut

[youtube] <http://bit.ly/Yoh4n>

Organisation

- 18h de CM — Y. BOICHUT (9h) et J. MARTIN (9h)
- 16h de TP — L. KALHEM et Y. BOICHUT

Le cours

- Introduction
- JUnit
- Tests fonctionnels
- Tests fonctionnels pour applications web
- Tests structurels
- Tests structurels pour applications web (Struts 2)
- Mocks
- Tests structurels de WS

Chapter 1. Introduction

1.1. Fail 1

- 22 juillet 1962 : la fusée qui portait la sonde Vénus Mariner I a dévié de sa trajectoire. Elle a du être détruite en vol
- Cause : un trait d'union oublié dans un programme Fortran
- Conséquences
 - Un programme de plus de 80 millions d'euros
 - Aucune perte humaine

1.2. Fail 2

- 1985-86 : la machine Therac 25 chargée de traiter des patients atteints du cancer leur a administré une dose mortelle de radiations. Lorsque le technicien tapait trop vite, le logiciel n'avait pas le temps de suivre et envoyait une très forte dose
- Cause : Hardware buffer overflow
- Conséquences : 5 décès

1.3. Fail 3

- 25 février 1991 : une batterie de missiles Patriot à Dashran échoue dans l'interception d'un missile Scud (687 mètres !!!)
- Cause : un bug dans l'arrondi du temps (en dixième de seconde) cumulé sur 100 heures
- Conséquences : 28 morts

1.4. Fail 4

- 1993 : résultats erronés retournés par le nouveau processeur Intel
- Cause : oubli du transfert d'une table de valeurs précalculées
- Conséquence : 475 millions de dollars

1.5. Fail 5

- 6 juillet 2012 : black-out du réseau de téléphonie
- Cause : défaillance logicielle HLR (Home localisation Register)
- Conséquences
 - 10 heures d'immobilisation du réseau. 26 millions de clients mobiles Orange.
 - Une estimation du 26 juillet 2012 s'élevait entre 30 et 40 millions d'euros

1.6. Fail 6 (1/2)

Ouvert 24h/24, fermé 7j/7

Le site du prix des carburants
Roulez éco

MINISTÈRE DE L'ÉCONOMIE
DES FINANCES ET DE L'INDUSTRIE

Nouvelle Recherche

Résultat de votre recherche : 7 points de vente

Commune	Nom du point de vente	Marque	Gazole	Mise à jour du Gazole	SP95	Mise à jour SP95
Dijon	CENTRE COMMERCIAL DIDON TOISON D'OR	CARREFOUR	0,996	26/12/2006	--	

Centre commercial Dijon Toison d'Or

Adresse : Route de Langres
Ville : 21000 Dijon
Horaires : Ouvert 24h/24
Jour(s) de Fermeture : Lundi Mardi Mercredi Jeudi Vendredi Samedi Dimanche
Service(s) proposé(s) : (À titre indicatif, sous la seule responsabilité du distributeur)

Dijon	ESSO BOURPOCHES	ESSO	0,989	03/01/2007	--	
Dijon	ESSO DIDON LANGRES	ESSO	0,990	30/12/2006	1,150	03/01/07
Dijon	ESSO SCHUMANN	ESSO	0,989	03/01/2007	--	
Dijon	RELAIS DES CONGRES	TOTAL	1,130	01/01/2007	1,260	01/01/07
Dijon	RELAIS GENOIS	TOTAL	1,130	31/12/2006	1,220	31/12/06
Fontaine les Dijon	FLORÉAL GEANT CASINO	GeANT	0,950	02/01/2007	1,150	02/01/07

AB CDEFGHIJKLMNOPQRSTUVWXYZ

Bug non corrigé à ce jour malgré publication dans SVM de mars 2007

Laurent BOCQUET, CNAM 2007

1.7. Fail 6 (2/2)

Votre carburant

Votre itinéraire

Foire aux questions

Comprendre les prix

Vos favoris

Contactez-nous

Liste des stations

Carte

➡ 13 points de vente triés par date de mise à jour

Trier les résultats par mise à jour du Gazole, ordre décroissant Appliquer

Il est également possible de trier en cliquant sur les entêtes des colonnes de la liste des résultats

Commune	Nom du point de vente	Marque	Gazole	Mise à jour du gazole	Ajout favoris	Plan
DIJON	CARREFOUR DIJON TOISON D OR	Carrefour	1.342	08/01/13		

CARREFOUR DIJON TOISON D OR

Carrefour

Route de Langres

21000 DIJON

Horaires

Ouvert 24h/24

Jour(s) de Fermeture :

Lundi Mardi Mercredi Jeudi Vendredi Samedi Dimanche

Service(s) proposé(s)

1.8. Fail 7 (1/5)

- 30 matchs à faire sur un we, des récompenses par paliers et des joueurs de la semaine à gagner dans des cartes rouges non vendables

16e semaine (jeudi 2 janvier 2020)



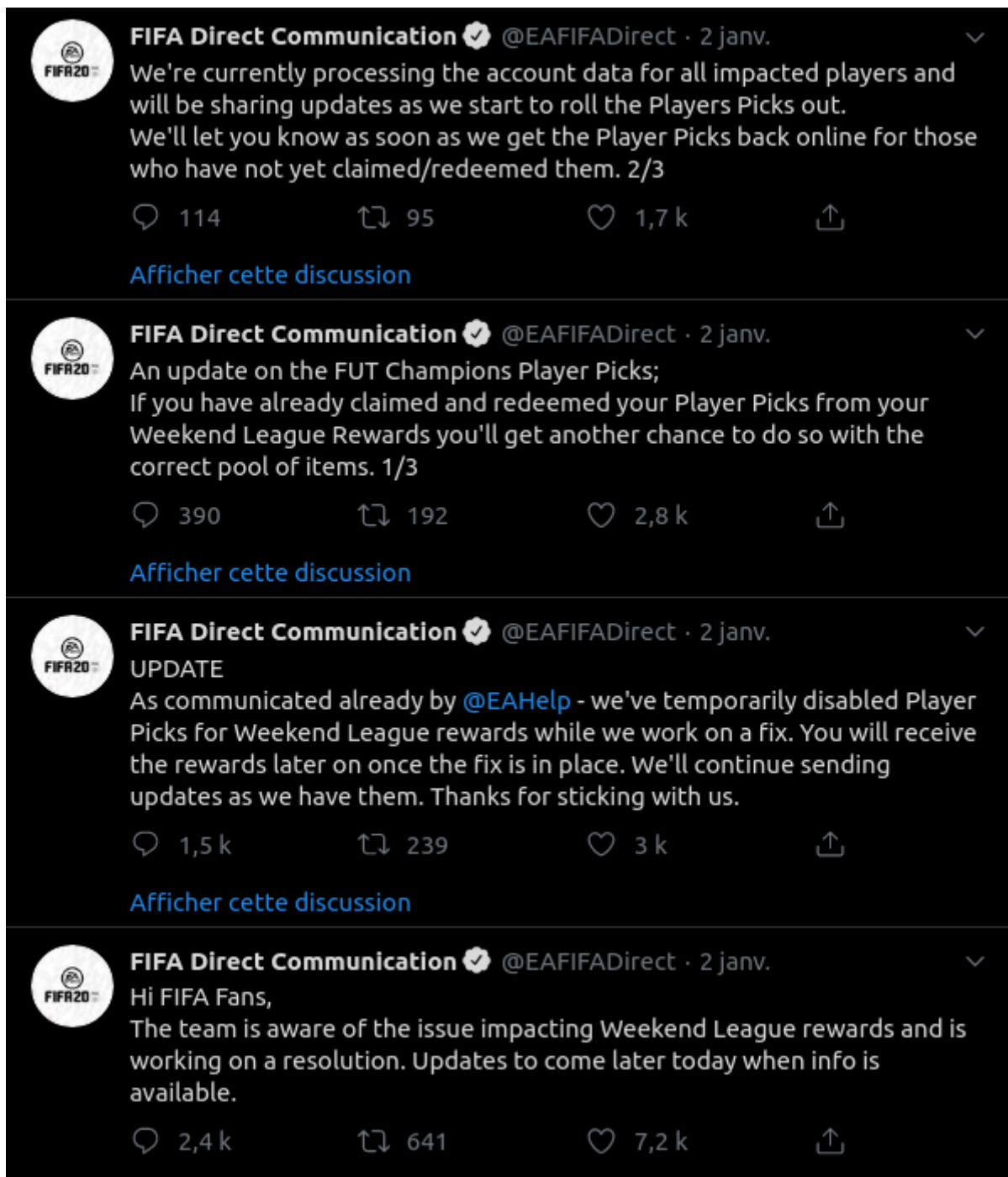
1.9. Fail 7 (2/5)



1.10. Fail 7 (3/5)



1.11. Fail 7 (4/5)



1.12. Fail 7 (5/5)

- Conséquences
 - Twitter en feu
 - YouTubers mécontents car contenus compromis pour leurs vidéos
 - Image EA encore un peu plus égratignée

1.13. Quand Konami s'y met aussi

Les promesses pour le 30 septembre 2021

- Free to play

- *The Simulation de football*



1.14. Quand Konami s'y met aussi

La réalité le 30 septembre 2021



1.15. Quand Konami s'y met aussi



1.16. Quand Konami s'y met aussi

- Jeu prévu le 11 novembre 2021
- Une semaine avant la date de sortie prévue, Konami annonce un report au printemps 2022
- Pire évaluation d'un jeu de tous les temps
- Remboursement des versions payantes immédiates après ce retentissant
- Des influenceurs spécialisés au bout de leur vie (MrQuinton, Luthor)

1.17. Le test... une activité simple ?

- Je teste le programme que j'ai écrit
 - Tests non objectifs ou du moins orientés

- Je teste le programme que **qqn d'autre** à écrit
 - Temps d'analyse

1.18. Motivations du test

1. **Erreur** dans la spécification, la conception ou le programme
2. **Défaut** dans le logiciel
3. **Défaillance** (ou **anomalie**) lors de l'exécution
4. **Coûts relatifs**
 - 60 milliards de dollars par an
 - 22 milliards pourraient être économisés si les procédures de test étaient améliorées

1.19. Qualité du logiciel (1/2)

- Tester s'inscrit dans une démarche de **qualité du logiciel**
- Techniques
 - **Validation** : est-ce que le logiciel réalise les fonctions attendues ?
 - **Vérification** : est-ce que le logiciel fonctionne correctement
- Le test est une technique de **validation**

1.20. Qualité du logiciel (2/2)

- **Test statique** : revues de code, de spécifications ou de documents de conception
- **Test dynamique** : exécution du code pour s'assurer d'un fonctionnement correct

1.21. Le test, what is it?

- Définition (IEE GSSET) : Le test est **l'exécution** ou l'évaluation d'un **système ou d'un composant** par des moyens automatiques ou manuels pour **vérifier qu'il répond à ses spécifications** ou identifier les **différences entre les résultats attendus et les résultats obtenus**
- Définition (Dijkstra) : Testing can **reveal the presence of errors but never their absence**

1.22. Test dynamique

Repose sur l'exécution du logiciel pour un sous-ensemble de données bien choisi du domaine de ses entrées possibles

Quatre activités

1. Sélection du jeu de tests
2. Exécution du jeu de test

3. Dépouillement des résultats

4. Evaluation de la qualité et de la pertinence des tests effectués (critère de l'arrêt)

1.23. Test dynamique : deux méthodes

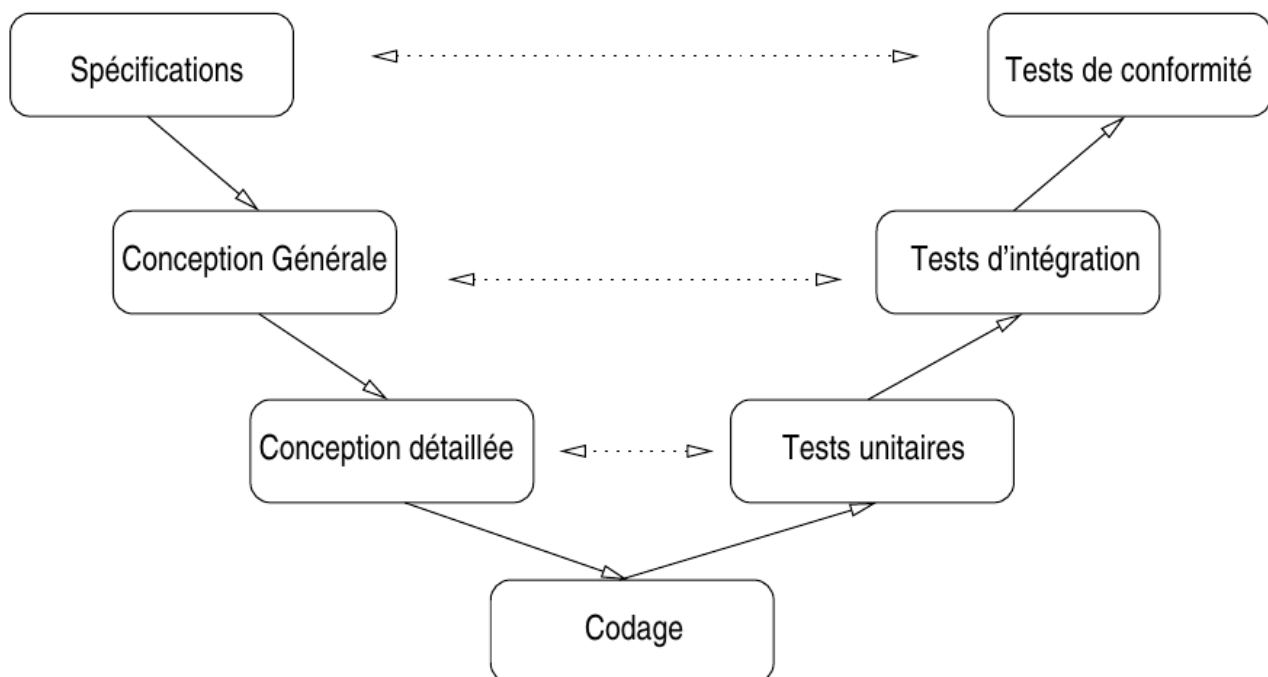
1. **Test fonctionnel** : jeu de tests sélectionné en s'appuyant sur les spécifications (boîte noire)

2. **Test structurel** : jeu de tests sélectionné en s'appuyant sur le code (boîte blanche)

1.24. Types de tests

- Tests unitaires
- Tests d'intégration
- Tests de recette/conformité
- Tests nominaux : test-to-pass
- Tests de robustesse : test-to-fail
- Test de performance : *load testing* et *stress testing*
- Tests de non régression

1.25. Test dans le cycle de vie du logiciel



1.26. Quelques références

- Le test des logiciels - Hermès 2000
- Les tests logiciels - Hermès 2011
- Software Engineering – Addison-Wesley, 6th ed. 2001. I. Sommerville

- Test logiciel en pratique – Vuibert Informatique, 2002. J. Watkins
- Quelques vidéos/tutos (Yoh4n) : https://www.youtube.com/playlist?list=PLYvwNvq6zdVHuCc-IHJNvO0_o1s-0sJrR

Chapter 2. JUnit

2.1. Tests unitaires

- Plateforme de test open source pour Java
 - Développée par Kent Beck et Erich Gamma
 - <http://www.junit.org>
 - <http://junit.sourceforge.net>
- Intégré à la majorité des IDEs Java
 - Eclipse, Netbeans, . . .

2.2. Vocabulaire JUnit

- **Assertion** : macro vérifiant à l'exécution qu'une condition est satisfaite
 - false → Exception + arrêt du test courant
- **Test fixture** : initialisation/terminaison commune à un ensemble de cas de test
- **Test Suite** : collection de cas de test
- **Classe sous test** : classe à tester
- **Classe de test** : classe regroupant les cas de test

2.3. Principes élémentaires

- Une dépendance Maven spécifiant la version de JUnit utilisée

```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.13</version>
  <scope>test</scope>
</dependency>
```

- Génération du src/test/java si nécessaire
 - Contiendra tous les jeux de tests
 - Architecture similaire en termes de packages que le src/main/java
- Exécution de mvn test pour exécuter toutes les classes de test

2.4. JUnit - programmation d'une classe de test

1. Associer à **chaque classe sous test** une (ou plusieurs) classe(s) de test
2. Une méthode annotée `@Test` pour **chaque cas de test** (les cas de tests doivent être

indépendants)

3. Scénario classique d'un cas de test

- a. Mise en place du contexte
- b. Appel de la méthode à tester
- c. Ecriture de l'oracle

4. Eventuellement ajouter des "Test fixtures" – une méthode d'initialisation ou de terminaison commune à tous les cas de test.

2.5. Assertions

- Importation statique des librairies nécessaires : `import static org.junit.Assert.*;`
- **Assertions disponibles**
 - `assertEquals(String msg, Object exp, Object act)`
 - `assertSame(String msg, Object exp, Object act)`
 - `assertEquals(String msg, Object[] exp, Object[] act)`
 - `assertTrue(String msg, boolean b)`
 - `assertNotNull(String msg, Object o)`
 - `fail(String msg)`
- Les **assertions** permettent d'effectuer l'évaluation du cas de test – Verdict KO/OK

2.6. Ce qu'il faut retenir : cas de test (1/3)

1. Mise en place du contexte
2. Appel de la méthode à tester
3. Définition de l'oracle du test

2.7. Ce qu'il faut retenir : fixtures (2/3)

- Opérations de préparation/nettoyage avant chaque cas de test
 - Initialisation commune pour tous les tests
 - Factorisation du code
 - Déclaration des variables globales
- Annotations `@Before` et `@After`
- JUnit invoque automatiquement ces méthodes avant (après) chaque méthode `@Test` (flot d'exécution)
- Démo : Exemple 1

2.8. Ce qu'il faut retenir : test-to-fail (3/3)

- Certains cas de tests sont corrects si une exception est levée
- Pas de blocs `try{...}catch{...}`
- Annotation paramétrée `@Test(expected=Exception.class)`
- Être précis dans l'exception attendue
- Une exception trop générale peut couvrir un dysfonctionnement

2.9. Mise en application sur l'écurie

- Nous gérons une écurie où on peut
 - ajouter un nouveau cheval (identifiant auto-incrémenté, nom, âge)
 - récupérer un cheval par son identifiant s'il existe
 - récupérer tous les chevaux
- Question : Définir les cas de tests intéressants
- Réponse : https://www.youtube.com/watch?v=c3j_rKIm4jE&list=PLYvwNvq6zdVHuCc-lHJNvO0_o1s-0sJrR&index=1&t=437s

2.10. Junit 5 — Jupiter

```
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.9.3</version>
    <scope>test</scope>
  </dependency>

  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-engine</artifactId>
    <version>5.9.3</version>
    <scope>test</scope>
  </dependency>

</dependencies>
```

2.11. Assertions plus précises

```
// permet de déclarer qu'un bloc d'instruction ne doit pas lever
// d'exception
Assertions.assertDoesNotThrow(
```

```
        ()->
        instance.voterReponse(idEtu,idQuestion,0));

// permet de déclarer précisément qu'une instruction ou bloc d'instructions
// doit lever une exception
Assertions.assertThrows(
    ADejaVoteException.class,
    ()->instance.voterReponse(idEtu,idQuestion,1));
```

2.12. Quelques annotations un peu différentes

```
// permet de préciser que la méthode m doit être exécutée avant
// chaque cas de test
@BeforeEach
public void m(){...}

// permet de préciser que la méthode m doit être exécutée après
// chaque cas de test
@AfterEach
public void m(){...}

// permet de préciser que la méthode m doit être exécutée au chargement
// de la classe
@BeforeAll
public void m(){...}

// permet de préciser que la méthode m doit être exécutée après
// que tous les tests aient été effectués
@AfterAll
public void m(){...}
```

Chapter 3. Tests fonctionnels

3.1. A tous les niveaux

- **Test fonctionnel** : valider le bon fonctionnement d'un logiciel, de méthodes de classes
- **Des tests unitaires** peuvent être utilisés pour faire du test fonctionnel i.e. à partir d'une interface commentée, nous pouvons préparer une liste de tests fonctionnels qui valideront les futures implémentations de cette interface
- **Test fonctionnel** : validation d'un logiciel i.e. nous avons une IHM et différents scénarios issus des **SFD** (spécifications fonctionnelles détaillées)
 - 1 scénario = 1 test fonctionnel
 - Scénario découpés en steps
 - On passe du step i au step i+1 si le step i est conforme à ce qu'on a précisé dans le test

3.2. Quality center (1/2)

Details

■ Design Steps

■ Parameters

Test Configurations

Attachments

Req Coverage

Linked I

Nom de l'étape	Description	Résultat attendu
Step 1	Objet : Lancer l'application "Affiliation" sur un environnement Pré-requis : Demander au chef de projet sur quel environnement le scénario doit être joué Données : Aucune Actions : 1) Double cliquer sur le répertoire de l'environnement ciblé 2) Ouvrir le répertoire "station PSCG" 3) Ouvrir le répertoire "Contrats" 4) Lancer la station "Affiliations"	1) Des répertoires par domaines sont disponibles 2) Les répertoires "Contrats", "Cotisations", "Modalité de gestion" et "Pilotage" s'affichent 3) Un répertoire "AURA" est disponible et les applications "Affiliations", "Compte-rendu d'affiliations", "Conditions générales" et "Station Contrats" sont disponibles 4) La fenêtre "Liste des périodes d'affiliations" s'ouvre.
Step 2	Objet : Rechercher un individu Pré-requis : Individu connu du référentiel personne	1) La fenêtre "Sélectionner un individu" s'affiche avec une liste d'individu ayant le nom <<<Nom>>>

3.3. Quality center (2/2)

Nom de l'étape	Statut	Date d'exec	Heure d'exec
Step 1	Passed	22/06/2016	16:28:55
Step 2	Passed	22/06/2016	16:28:56
Step 3	Passed	22/06/2016	16:28:56
Step 4	Passed	22/06/2016	16:28:57
Step 5	Failed	22/06/2016	16:28:58
Step 6	No Run	22/06/2016	16:28:52
Step 7	No Run	22/06/2016	16:28:52
Step 8	No Run	22/06/2016	16:28:52

Description

Objet :
Vérification de la fenetre "Liste des périodes d'affiliation"

Pré-requis :
Aucun

Données :
Aucune

Actions :
1)Vérifier la présence des 3 blocs :
- Bloc Participant
- Bloc Période de carrière interne
- Bloc des périodes d'affiliation

Résultat attendu:
1) Les 3 blocs sont bien présents

Résultat obtenu:
Il n'y a que 2 blocs. Il manque le bloc "Périodes d'affiliation"

3.4. Test fonctionnel

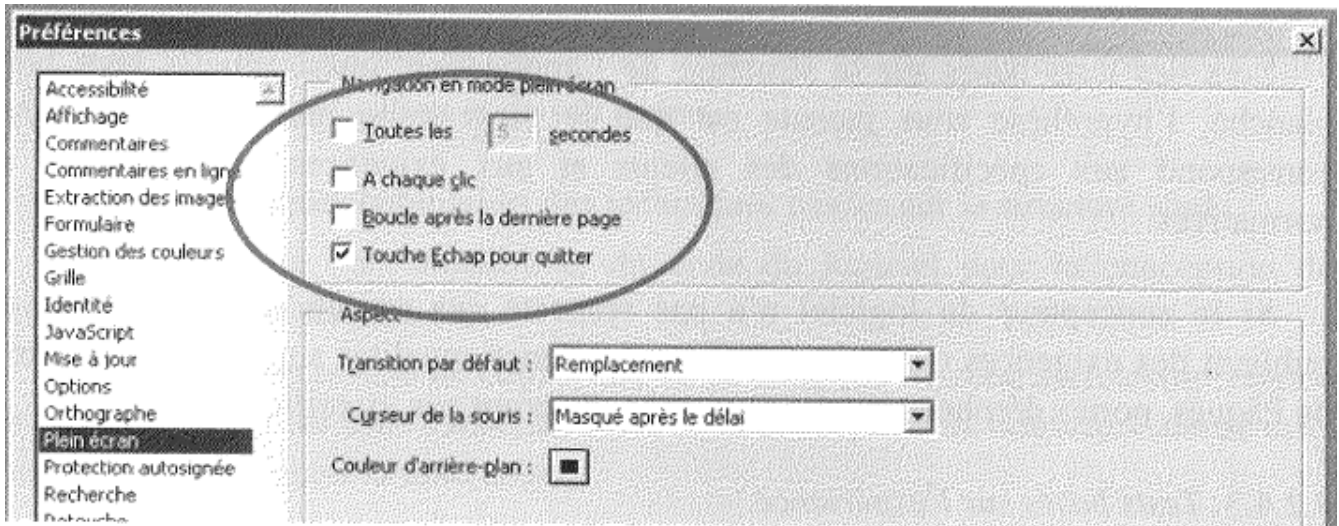
- La théorie voudrait qu'on écrive les tests fonctionnels indépendamment des implémentations
- C'est faisable sur tout ce qui est IHM
- Plus compliqué, quand on veut tester fonctionnellement une classe
- Application de la théorie aux classes → test d'intégration

3.5. Approche du test fonctionnel

Cas classique

- On a un écran avec des champs
- On doit choisir des valeurs pour tester les scénarios envisagés
- On observe ce que l'on est censé observer selon les valeurs choisies
- Et tout ceci indépendamment de la façon dont la chose a été codée

3.6. Les problématiques liées au test fonctionnel



- Combien y a t'il de tests à faire si nous voulons être exhaustif ?
- Première estimation : $2^8 * 100 = 1600$
- Dans le champ on peut taper deux caractères : $2^8 * 65536^2 = 68\,719\,476\,736$ cas
- Au rythme d'un test par seconde : 2177,5 ans pour exécuter l'ensemble des tests...

3.7. Partitions d'équivalence

- But : réduire le nombre de tests sans pour autant perdre en efficacité
- Principe : toutes les valeurs d'une même partition passeront par les mêmes instructions et seront traitées de la même façon
- Un représentant par classe d'équivalence

3.8. Exemple de partitions d'équivalence

$$\sqrt{\frac{1}{x}}$$

- C0 : $x < 0$
- C1 : $x = 0$
- C2 : $x > 0$

3.9. Combinaison des partitions d'équivalence

- Pour chaque paramètre, il faut établir les partitions d'équivalences
- Combinaisons de toutes les partitions pour l'ensemble des paramètres pour être *exhaustif*
- Cependant un seul cas d'erreur à la fois de manière générale

3.10. Aux limites il se passe toujours des choses bizarres

- Une bonne pratique vise à tester aux limites des plages des valeurs
- Application sur l'exemple $\sqrt{1/x}$
 - Que se passe t'il juste avant 0 ?
 - Que se passe t'il juste après 0 ?

3.11. Limiter le nombre de combinaisons

- Même si on utilise les classes d'équivalence, le nombre de tests peut parfois être trop élevé
- Utilisation d'approches visant à limiter le nombre de tests tout en garantissant certaines propriétés :
 - Approche PairWise
 - Approche nWise

3.12. Approche PairWise

- Choix des valeurs guidé pour garantir que chaque combinaison de 2 variables est testée
- Pourquoi un tel choix ?
- Statistiquement parlant, la majorité des fautes est détectée par des combinaisons de deux valeurs
- Exemple d'un générateur : <https://pairwise.teremokgames.com/>
- Liste de générateurs : pairwise.org

3.13. Exemple de l'approche PairWise

- Soit 4 variables os, réseau, imprimante et application telles que
 - os : {XP, Linux, MAC}
 - réseau : {ATM, Wifi, Bluetooth}
 - imprimante : {HP35, Canon900, CanonEX}
 - application : {Word, Excel, Power Point}
- Toutes les combinaisons : 81 possibilités

- Approche pairwise : 9 combinaisons seulement (démon)

3.14. Tests fonctionnels à partir d'interfaces

- Apprentissage sur une étude de cas (réseau social)
- Interface = SFD
- Généralisation des tests fonctionnels à toute implémentation : tests d'intégration

3.15. Approche fonctionnelle pour le test de classes

Raisonnement pour tester une fonction

- Identification des paramètres et des valeurs possibles
- Identifier les représentants canoniques des classes d'équivalence
- Calculer les combinaisons nécessaires
- Associer aux combinaisons les résultats attendus
- Tester

3.16. Réseau social (1/4)

```
/**
 * Méthode pour l'inscription
 * @param pseudo
 * @param nom
 * @param prenom
 * @param password
 * @throws LoginDejaPrisException : le pseudo choisi est déjà
 * pris par un autre utilisateur
 * @throws InformationManquanteException : il manque
 * une des informations lors de l'inscription
 */
void inscription(String pseudo, String nom, String prenom,
                String password)
    throws LoginDejaPrisException,
           InformationManquanteException;
```

3.17. Réseau social : raisonnement (2/4)

- Identification des classes d'équivalence
 - pseudo : {null, "", "toto"}
 - nom : {null, "", "Toto"}
 - prenom : {null, "", "Tata"}

- password : {null,"","tutu"}
- Inscription avec toutes les données renseignées et utilisateur inconnu par le système
- Inscription avec toutes les données renseignées et utilisateur connu par le système
- Cas d'erreur : 8 cas (pour chaque champ, 2 cas d'erreur)

3.18. Réseau social : implémentation des cas de tests (3/4)

```
public class TestFacadeChatImpl {
    FacadeChatImpl facadeChatRoom;
    @Test
    public void inscriptionClassique() {
        try {
            facadeChatRoom.inscription("toto", "Toto",
                                       "tata", "tutu");
            // assertions pour vérifier la bonne inscription
        } catch (LoginDejaPrisException e) {
            Assert.fail();
        } catch (InformationManquanteException e) {
            Assert.fail();
        }
    }
    ...
}
```

3.19. Les astuces de sioux

- Important d'avoir une structure miroir entre l'organisation packages côté test et l'organisation packages côté source
- Privilégié la visibilité **package** pour vos champs de classe
- **Pourquoi ?**
- Car lors de l'exécution de vos tests, **grâce à l'organisation miroir**, vos classes de tests ont un accès **direct** aux **champs**

3.20. Application pour les assertions du cas de test

```
package facade;
public class TestFacadeChatImpl {
    FacadeChatImpl facadeChatRoom;
    @Test
    public void inscriptionClassique() {
        try {
            facadeChatRoom.inscription("toto", "Toto",
```

```

        "tata", "tutu");
    Personne p = facadeChatRoom.personnesInscrites
        .get("toto");
    Assert.assertNotNull(p);
    Assert.assertEquals(p.getNom(), nom);
    Assert.assertEquals(p.getPrenom(), prenom);
    Assert.assertEquals(p.getPassword(), mdp);
} catch (LoginDejaPrisException e) {Assert.fail();}
} catch (InformationManquanteException e) {Assert.fail();}
}
}
}

```

3.21. Réseau social : Bilan (4/4)

- Exemple de tests unitaires à vocation fonctionnelle
- Nous avons attendu l'application pour écrire nos tests unitaires. Sont ils dépendants d'une implémentation ?
- Peut-on faire mieux que ça ?
- Refactoring : classes abstraites (jeux de tests) et utilisation de fabriques (pour l'instance à tester).
- **Spoiler** : une fabrique est une classe qui permet de créer des instances. Voir cours de MAC S2.

3.22. Test fonctionnel → Test d'intégration

- Définir une classe de test abstraite permettant de manipuler une instance quelconque de l'interface visée
- Ecrire tous les cas de tests attendus
- Déléguer à l'aide de méthodes abstraites les verdicts quand ceux ci ne sont pas exprimables de manière générale
- Utilisation de fabriques pour les instances à tester

3.23. Classe de test abstraite

```

public abstract class TestIntegrationFacadeChatRoom {
    protected FacadeChatRoom facadeChatRoom;
    private FabriqueFacadeChatRoom fabriqueFacadeChatRoom;
    public TestIntegrationFacadeChatRoom(
        FabriqueFacadeChatRoom fabriqueFacadeChatRoom) {
        this.fabriqueFacadeChatRoom = fabriqueFacadeChatRoom;
    }
    @Before
    public void reinitialisationFacade() {
        facadeChatRoom = fabriqueFacadeChatRoom.getFacade();
    }
}

```

```

    }
    public abstract void oracleTestInscriptionOK(
        String pseudo, String nom, String prenom, String mdp);

    @Test
    public void testInscriptionOK() throws LoginDejaPrisException,
        InformationManquanteException {
        facadeChatRoom.inscription("Yoh","Boichut","yohan","mdp");
        oracleTestInscriptionOK("Yoh","Boichut","yohan","mdp");
    }
}

```

3.24. Classe de test abstraite

- Gestion de l'instance courante à gérer
- Nécessite une fabrique d'instances pour obtenir l'instance courante
- Délègue la vérification des assertions à la future fille si impossibilité d'écrire complètement les assertions
- Ecriture générique des cas de tests

3.25. Classe de test fille

```

public class TestIntegrationFacadeChatRoomImpl
    extends TestIntegrationFacadeChatRoom {
    public TestIntegrationFacadeChatRoomImpl() {
        super(new FabriqueFacadeChatRoomImpl());
    }
    @Override
    public void oracleTestInscriptionOK(
        String pseudo, String nom,
        String prenom, String mdp) {
        Personne personne = ((FacadeChatRoomImpl)
            this.facadeChatRoom).personnesInscrites.get(pseudo);
        Assert.assertNotNull(personne);
        Assert.assertEquals(personne.getNom(),nom);
        Assert.assertEquals(personne.getPrenom(),prenom);
        Assert.assertEquals(personne.getPassword(),mdp);
    }
}

```

3.26. Classe de test fille

- Implémentation du constructeur vide qui alimente le constructeur prenant la fabrique d'instances pour la classe mère
- Implémentation des différentes méthodes de **verdict** abstraites

3.27. Un peu de recul sur la méthodologie

- Nous avons pensé nos tests indépendamment du code réel derrière : **test boîte noire**
- Nous avons pris un test représentant par type de comportement : **classes d'équivalences**
- Limitation du nombre de cas de tests par l'approche PairWise
- Généralisation des **tests fonctionnels** à toute implémentation : **tests d'intégration**

3.28. Compétences à acquérir

- Ecrire des tests unitaires structurés, commentés - JUNIT
- Ecrire des tests indépendamment des implémentations - JUNIT → Test d'intégration
- Savoir distinguer les uses cases (positifs et négatifs)
- Savoir définir les données pertinentes pour les cas de tests
- Savoir limiter les jeux de données

Chapter 4. Tests fonctionnels — Applications Web

4.1. Pour une IHM donnée, ...

- **Use cases** comme pour les interfaces avec **choix pertinents des scénarios**
- Utilisation de **Selenium pour définir nos tests**
- **Bonnes pratiques** pour l'utilisation de Selenium
- A développer :
 - extraction en code Java et
 - possibilité de rejouer les tests à partir de tests unitaires

4.2. Selenium

- Plugin FireFox/Chrome
- Permet d'**enregistrer des scénarios** de navigation en enregistrant l'interaction humain/Appli
- Permet de définir des "**assertions**" i.e. choses à vérifier (présence de texte, d'une valeur, etc.)
- Nécessité de **bien identifier** les composants de l'application web grâce à l'attribut **id** dans les composants HTML
- Ne **JAMAIS** localiser un composant par son XPath... (anecdote)

4.3. Selenium vs Paris en ligne

Use case : annulation d'un pari déjà terminé

Scénario

1. Connexion avec yo/yo
2. Sélection dans le menu de l'affichage des paris déjà réalisés
3. Demander l'annulation du pari France vs Australie
4. Constater que l'annulation est impossible
5. Se déconnecter

4.4. Selenium et automatisation

- Le plugin chrome cool pour un petit dév
- Difficile d'industrialiser le concept dans l'état
- Solution : Selenium Grid

4.5. Un petit point d'histoire en premier lieu

- Avant l'avènement de Selenium Grid → Junit à partir du plugin
- Très compliqué à mettre en place car
 - Webdriver dépendant du navigateur

4.6. Selenium Grid

- Selenium Hub
 - Contient des noeuds i.e. instances avec navigateur dédié
 - Tests exécutés en // sur chacune des instances
- Utilisation de docker pour déployer
 - Selenium Hub
 - Un noeud Chrome
 - Un serveur d'application

4.7. Un exemple pratique ?

- See → <https://youtu.be/oyMt9ob9RBQ>
- Live démo pour illustrer les choses à savoir

Chapter 5. Tests structurels

5.1. Principes fondamentaux

- Le **test structurel** s'appuie sur le **code** pour **déterminer les différents jeux de tests**
- Les jeux de tests sont aussi faits en fonction d'un **critère de couverture**
- **Aucun intérêt** de savoir à quoi sert la fonction
- Le but est de détecter **des exceptions à l'exécution** (pointeur nul, dépassement de limites d'un tableau, etc)
- **En passant partout**, on a des chances de lever ce genre de bug

5.2. Critères de couverture

En considérant le programme comme un graphe

- **Tous-les-noeuds** : chaque instruction est considérée comme un noeud. Il faut donc élaborer un ensemble de jeux de test permettant de passer par **toutes les instructions**
- **Tous-les-arcs**
 - Le but de jeu est de passer dans toutes les **branches conditionnelles** (if, switch, for, while)
 - Si **tous-les-arcs** est satisfait, alors **tous-les-noeuds** l'est également
- **Toutes-les-conditions** : dans le cas de conditions multiples, elles doivent toutes prendre les valeurs vrai/faux

5.3. Impact au niveau des jeux de données

- Plus le **critère est élevé** dans l'échelle des critères, plus le **nombre de jeux de test** est **grand**
- Plus le **nombre de jeux de test** est **grand**, plus la **confiance s'accroît** dans le bon fonctionnement
- Le critère couramment rencontré : **tous-les-arcs**
- Pour les **systèmes critiques** (en aéronautique), **toutes les combinaisons de valuations de sous-conditions** doivent faire l'objet de jeux de tests

5.4. Toutes les instructions

```
class A<X>{  
    private X[] tab;  
    public A(X[] t) {  
        this.tab = t;  
    }  
    public int find(X e) {  
        int i = 0;
```

```

    int resultat = -1;
    boolean trouve = false;
    while (!trouve){
        if (this.tab[i].equals(e) && i<= tab.length) {
            resultat = i;
            trouve = true;
        }
        i++;
    }
    return resultat;
}
}

```

Il suffit d'un jeu de test pour satisfaire le critère avec $t = [2]$ et on recherche 2

5.5. Tous-les-arcs

```

class A<X>{
    private X[] tab;
    public A(X[] t) {
        this.tab = t;
    }
    public int find(X e) {
        int i = 0;
        int resultat = -1;
        boolean trouve = false;
        while (!trouve){
            if (this.tab[i].equals(e) && i< tab.length) {
                resultat = i;
                trouve = true;
            }
            i++;
        }
        return resultat;
    }
}

```

Il suffit d'un jeu de test pour satisfaire le critère avec $t = [0;2]$ et on recherche 2

5.6. Et le bug ?

```

class A<X>{
    private X[] tab;
    public A(X[] t) {
        this.tab = t;
    }
    public int find(X e) {

```

```

    int i = 0;
    int resultat = -1;
    boolean trouve = false;
    while (!trouve){
        if (this.tab[i].equals(e) && i < tab.length) {
            resultat = i;
            trouve = true;
        }
        i++;
    }
    return resultat;
}
}

```

5.7. SonarQube via Docker (1/2)

- Instance docker
- Rien à installer (sinon Docker)

```

## Permet de récupérer l'image latest
docker pull sonarqube
## Permet de lancer l'instance avec mapping de port sur localhost 9000
docker run -p 9000:9000 -d sonarqube

```

5.8. Configuration SonarQube

- Lancer localhost:9000 et se connecter en admin
 - login : admin
 - mdp : admin
- Cliquer sur **Administration** → **Security** → **Users**
- Pour l'utilisateur **Administrator**, dans la colonne **Tokens**, cliquer sur le petit menu **update tokens**
- Donnez un nom au token et copier/coller le token généré

5.9. Dans votre projet Maven 1/

```

<!-- sonar.login est le token que vous avez généré-->
<properties>
    <sonar.login>squ_237bea46530231801097e2a76c718e81ba1dc3d2</sonar.login>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
</properties>

```

5.10. Dans votre projet Maven 2/

Dans build/plugins

```
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.7</version>
  <executions>
    <execution>
      <id>prepare-agent</id>
      <goals>
        <goal>prepare-agent</goal>
      </goals>
    </execution>
    <execution>
      <id>report</id>
      <goals>
        <goal>report</goal>
      </goals>
      <configuration>
        <formats>
          <format>XML</format>
        </formats>
      </configuration>
    </execution>
  </executions>
</plugin>
<plugin>
  <groupId>org.sonarsource.scanner.maven</groupId>
  <artifactId>sonar-maven-plugin</artifactId>
  <version>3.8.0.2131</version>
</plugin>
```

5.11. Dans votre projet Maven 2/

Pour lancer l'analyse :

```
mvn clean verify sonar:sonar
```

5.12. Conclusion

- Test fonctionnel et test structurel sont deux **méthodologies complémentaires**
- **Aucune ne peut garantir l'absence de bug**
- La **qualité des tests** structurels **dépend** essentiellement du **jeu de données** plus ou moins orienté par le critère visé

- Prendre un peu de recul vis à vis du critère visé pour **choisir les meilleurs données**

Chapter 6. Les Mocks

6.1. Concept

- Les mocks sont des objets simulés qui peuvent être utilisés pour remplacer les dépendances d'une classe ou d'un objet dans les tests unitaires.
- Les mocks permettent de contrôler et de tester les comportements de certaines dépendances afin d'affiner les tests unitaires et de garantir la qualité du code.

6.2. Motivations

- Pour tester les algorithmes complexes
- Pour tester des fonctionnalités critiques
- Pour améliorer les performances des tests
- Pour isoler simplement la classe sous test du reste du monde

6.3. Exemple de cas intéressant

```
public class Calculator {  
    private MathUtils mathUtils;  
  
    public Calculator(MathUtils mathUtils) {  
        this.mathUtils = mathUtils;  
    }  
  
    public int calculateSum(int a, int b) {  
        return mathUtils.add(a, b);  
    }  
}
```

On veut tester la classe **Calculator** → On procède comment ?

6.4. Processus

- On veut tester **Calculator** en isolation
- On a besoin de simuler les instances de **MathUtils**
- Le but est donc de définir des tests unitaires ou fonctionnels tout en simulant ce qui n'est pas **Calculator** et qui n'est pas un objet issu de la JDK

6.5. Ce qu'on ne veut pas voir

```
public class Calculator {
```

```

private MathUtils mathUtils;

public Calculator(MathUtils mathUtils) {
    this.mathUtils = mathUtils;
}

public int calculateSum(int a, int b) {
    return mathUtils.add(a, b);
}
}

```

- Utiliser un Mock pour Calculator et enregistrer le scénario suivant : si j'appelle calculateSum avec 3 et 4 alors il doit me retourner 7.
- J'appelle la fonction calculateSum avec 3 et 4. Surprise !!! J'obtiens 7

6.6. Ce qu'on attend

```

public class Calculator {
    private MathUtils mathUtils;

    public Calculator(MathUtils mathUtils) {
        this.mathUtils = mathUtils;
    }

    public int calculateSum(int a, int b) {
        return mathUtils.add(a, b);
    }
}

```

- Utiliser un Mock pour MathUtils et enregistrer le scénario suivant : si j'appelle add avec 3 et 4 alors il doit me retourner 7.
- Je crée un objet Calculator en injectant le mock
- J'appelle la fonction calculateSum avec 3 et 4. Surprise !!! J'obtiens 7

6.7. On teste des Mocks ?

- SURTOUT PAS !!!!
- Pourtant je le vois tous les ans
- Par contre on teste des **mocks partiels**
- Plus précisément, on teste une méthode isolée grâce aux **mocks partiels**

6.8. Etapes indispensables pour utiliser des Mocks

1. Création du Mock

2. Définition des appels prévus et des résultats attendus
3. Mise en service du mock
4. Utilisation

6.9. Mocking partiel

- Le mocking partiel en test unitaire consiste à remplacer seulement une partie des dépendances d'une classe ou d'un objet avec des objets simulés (mocks)
- Permet de se concentrer sur le comportement spécifique de la classe ou de l'objet testé, sans tenir compte du comportement des autres dépendances.

6.10. Mocking partiel cas extrême

- Cela permet de tester une méthode en particulier en mockant ses dépendances
- Garantissant ainsi que les résultats renvoyés par la méthode sont corrects sans tenir compte du comportement des autres méthodes et dépendances.

6.11. Exemple de cas intéressant

```
public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }
    public int multiply(int a, int b) {
        return a * b;
    }
    public int calculateSumOfSquares(int a, int b) {
        int squareA = multiply(a, a);
        int squareB = multiply(b, b);
        return add(squareA, squareB);
    }
}
```

- Comment tester **calculateSumOfSquares** ? → Tester **add** et **multiply** en premier ?
- Non. Définir un mock partiel de Calculator en ignorant les comportements de **add** et **multiply**
- En d'autres termes : **mocking des 3 appels internes**

Chapter 7. Mockito

7.1. Introduction

- Les mocks sont souvent utilisés dans les tests unitaires
 - pour simuler les dépendances
 - vérifier les comportements de certaines parties d'une application.
- La bibliothèque Mockito est une bibliothèque populaire pour créer des mocks en Java.

7.2. Installation de Mockito

Pour utiliser Mockito dans votre projet, vous devez d'abord l'ajouter en tant que dépendance à votre fichier pom.xml

```
<dependency>
  <groupId>org.mockito</groupId>
  <artifactId>mockito-core</artifactId>
  <version>3.3.3</version>
  <scope>test</scope>
</dependency>
```

7.3. Installation de Mockito avec Junit5

Pour utiliser Mockito dans votre projet, vous devez d'abord l'ajouter en tant que dépendance à votre fichier pom.xml

```
<dependency>
  <groupId>org.mockito</groupId>
  <artifactId>mockito-junit-jupiter</artifactId>
  <version>5.3.1</version>
  <scope>test</scope>
</dependency>
```

7.4. Création de mocks

Pour créer un mock avec Mockito, vous pouvez utiliser la méthode Mockito.mock(). Par exemple, pour créer un mock d'une interface List:

```
List mockList = mock(List.class);
```

Possibilité de créer un mock à la déclaration du champ avec **@Mock**

7.5. Configuration de comportements de mocks

Vous pouvez définir le comportement d'un mock en utilisant des instructions `when` et `thenReturn`. Par exemple, pour définir le comportement d'une méthode `get` pour retourner une valeur par défaut:

```
when(mockList.get(0)).thenReturn("Premier élément");
```

7.6. Vérification de comportements de mocks

Pour vérifier que les méthodes sont appelées sur un mock, vous pouvez utiliser la méthode `verify` de Mockito. Par exemple, pour vérifier que la méthode `get` a été appelée une fois:

```
verify(mockList).get(0);
```

7.7. Mocking partiel avec Mockito

```
public class Calculator {
    public int multiply(int a, int b) {
        return a * b;
    }

    public int add(int a, int b) {
        return a + b;
    }

    public int calculateSumOfSquares(int a, int b) {
        int squareOfA = multiply(a, a);
        int squareOfB = multiply(b,b);
        return add(squareOfA,squareOfB);
    }
}
```

7.8. Test de la classe précédente

```
@RunWith(MockitoJUnitRunner.class)
public class CalculatorTest {

    @Spy
    private Calculator calculator;

    @Test
    public void testCalculateSumOfSquares() {
        int a = 2;
```

```

    int b = 3;
    int squareA = 4;
    int squareB = 9;
    int expectedSum = squareA + squareB;
    doReturn(squareA).when(calculator).multiply(a, a);
    doReturn(squareB).when(calculator).multiply(b, b);
    doReturn(expectedSum).when(calculator).add(squareA, squareB);
    int actualSum = calculator.calculateSumOfSquares(a, b);
    assertEquals(expectedSum, actualSum);
}
}

```

7.9. Attention avec les annotations

- Si vous n'utilisez pas **@RunWith** les mocks ou mocks partiels ne sont alors pas automatiquement instanciés
- Il faut utiliser l'instruction **MockitoAnnotations.initMocks(this)**
- Attention avec Junit 5, il faut **@ExtendWith(MockitoExtension.class)**

7.10. Alternative

Il est aussi possible de ne pas utiliser l'annotation **@Spy** ou **@Mock**

```

public class CalculatorTest {
    private Calculator calculator;
    @Before
    public void setup() {
        calculator = spy(new Calculator());
    }

    @Test
    public void testCalculateSumOfSquares() {
        int a = 2;
        int b = 3;
        int squareA = 4;
        int squareB = 9;
        int expectedSum = squareA + squareB;
        doReturn(squareA).when(calculator).multiply(a, a);
        doReturn(squareB).when(calculator).multiply(b, b);
        doReturn(expectedSum).when(calculator).add(squareA, squareB);
        int actualSum = calculator.calculateSumOfSquares(a, b);
        assertEquals(expectedSum, actualSum);
    }
}

```

7.11. Là où ça devient obscure...

- Attention le test qui va suivre n'est pas un test digne de ce nom
- Des étudiants ont eu 0 pour ce genre de "test"

7.12. Le côté obscure de Spy 1/

```
public class UserService {
    private UserDao userDao;
    public boolean isUserAdmin(String userId) {
        User user = userDao.getUserById(userId);
        if (user != null && user.isAdmin()) {
            return true;
        }
        return false;
    }
    public void setUserDao(UserDao userDao) {
        this.userDao = userDao;
    }
}
```

7.13. Le côté obscure de Spy 2/

```
public interface User {
    boolean isAdmin();
}

public interface UserDao {
    User getUserById(String userId);
}
```

7.14. Le côté obscure de Spy 3/

```
public class UserServiceTest {
    @Test
    public void testIsUserAdmin() {
        UserDao userDao = Mockito.mock(UserDao.class);
        UserService userService = new UserService();
        userService.setUserDao(userDao);

        User user = Mockito.mock(User.class);
        Mockito.when(userDao.getUserById("123")).thenReturn(user);
        UserService spyUserService = Mockito.spy(userService);
        Mockito.doReturn(true).when(spyUserService).isUserAdmin("123");
        assertTrue(spyUserService.isUserAdmin("123"));
    }
}
```

```

        // Que va t'il se passer après l'exécution de l'instruction
        // ci-dessous ??
        System.out.println(spyUserService.isUserAdmin("456"));
    }
}

```

7.15. Eclairages du côté obscur de Spy et Mock

- Contrairement à certaines bibliothèques comme EasyMock, il n'y a pas d'erreur par défaut si un appel imprévu a lieu
- En clair si un appel imprévu a lieu sur
 - un mock : valeur par défaut retournée (null pour tout objet, false pour booléens, etc.)
 - un spy : exécution du code réel
- Utilisation de **Mockito.verifyNoMoreInteractions(<Mocks,Spies>)**

7.16. Mocking — HOW TO avec méthode

- Pour un Mock : rien à faire de particulier sauf évidemment si vous voulez vérifier qu'uniquement les scénarios prévus ont eu lieu
- Pour un Spy : indispensable de le spécifier car sinon c'est le code réel qui est exécuté

7.17. En pratique

```

// Création du mock
SomeClass someMock = mock(SomeClass.class);

// Prédiction de l'appel de méthode void
doNothing().when(someMock).someVoidMethod();

// Appel de la méthode sur le mock
someMock.someVoidMethod();

// Vérification que la méthode a été appelée une seule fois
verify(someMock, times(1)).someVoidMethod();

```

Pour un **Spy**, c'est exactement la même chose

7.18. Spécifier une levée d'exception

```

// Levée d'une exception pour l'appel d'une fonction/méthode
// sur un mock/spy
doThrow(MonException.class).when(mock).m();

```

7.19. Petit cas d'étude

```
public class UserService {

    private final UserServiceRepository repository;
    private final EmailService emailService;

    public UserService(UserServiceRepository repository, EmailService emailService) {
        this.repository = repository;
        this.emailService = emailService;
    }

    public void createUser(User user) {
        if (user == null || user.getName() == null || user.getEmail() == null) {
            throw new IllegalArgumentException("User must have name and email");
        }
        repository.save(user);
    }

    public List<User> searchUsers(String query) {
        if (query == null || query.trim().isEmpty()) {
            throw new IllegalArgumentException("Query cannot be null or empty");
        }
        return repository.search(query);
    }

    public void sendEmailToAllUsers(String subject, String body) {
        List<User> users = repository.getAllUsers();
        for (User user : users) {
            emailService.sendEmail(user.getEmail(), subject, body);
        }
    }
}
```

7.20. Classe de test bien faite

```
public class UserServiceTest {

    @Mock
    private UserServiceRepository repositoryMock;

    @Mock
    private EmailService emailServiceMock;

    @Spy
    private UserService userServiceSpy;

    @Before
```

```

public void setUp() {
    MockitoAnnotations.openMocks(this);
    userServiceSpy = new UserService(repositoryMock, emailServiceMock);
}

@Test(expected = IllegalArgumentException.class)
public void testCreateUserWithNullUser() {
    userServiceSpy.createUser(null);
}

@Test(expected = IllegalArgumentException.class)
public void testCreateUserWithNullName() {
    userServiceSpy.createUser(new User(null, "test@test.com"));
}

@Test(expected = IllegalArgumentException.class)
public void testCreateUserWithNullEmail() {
    userServiceSpy.createUser(new User("Test", null));
}

@Test
public void testCreateUserSuccess() {
    User user = new User("Test", "test@test.com");
    userServiceSpy.createUser(user);
    verify(repositoryMock).save(user);
}

@Test
public void testSendEmailToAllUsers() {
    List<User> users = new ArrayList<>(Arrays.asList(
        new User("Test1", "test1@test.com"),
        new User("Test2", "test2@test.com"),
        new User("Test3", "test3@test.com")
    ));
    when(repositoryMock.getAllUsers()).thenReturn(users);
    userServiceSpy.sendEmailToAllUsers("Test Subject", "Test Body");
    for (User user : users) {
        verify(emailServiceMock).sendEmail(user.getEmail(), "Test Subject", "Test Body"
    );
    }
}

@Test
public void testSendEmailToAllUsersWithNoUsers() {
    when(repositoryMock.getAllUsers()).thenReturn(new ArrayList<>());
    userServiceSpy.sendEmailToAllUsers("Test Subject", "Test Body");
    verify(emailServiceMock, never()).sendEmail(anyString(), anyString(), anyString()
    );
}
}

```

7.21. Cas complexes

- Parfois, vous devez vérifier des appels de méthode avec des arguments spécifiques qui peuvent être difficiles à prévoir (création avec un **new**).
- Mockito fournit des **ArgumentMatchers** pour gérer ces situations.

7.22. Les ArgumentMatchers (1/3)

Matcher	Description
<code>any()</code>	Correspond à n'importe quelle valeur d'un type générique.
<code>any(Class<T> clazz)</code>	Correspond à n'importe quelle instance du type spécifié.
<code>anyString/Int/Long/List/Map/Set/Boolean()</code>	N'importe quel(le) String/entier/long/liste/map/set/booléen.

7.23. Les ArgumentMatchers (2/3)

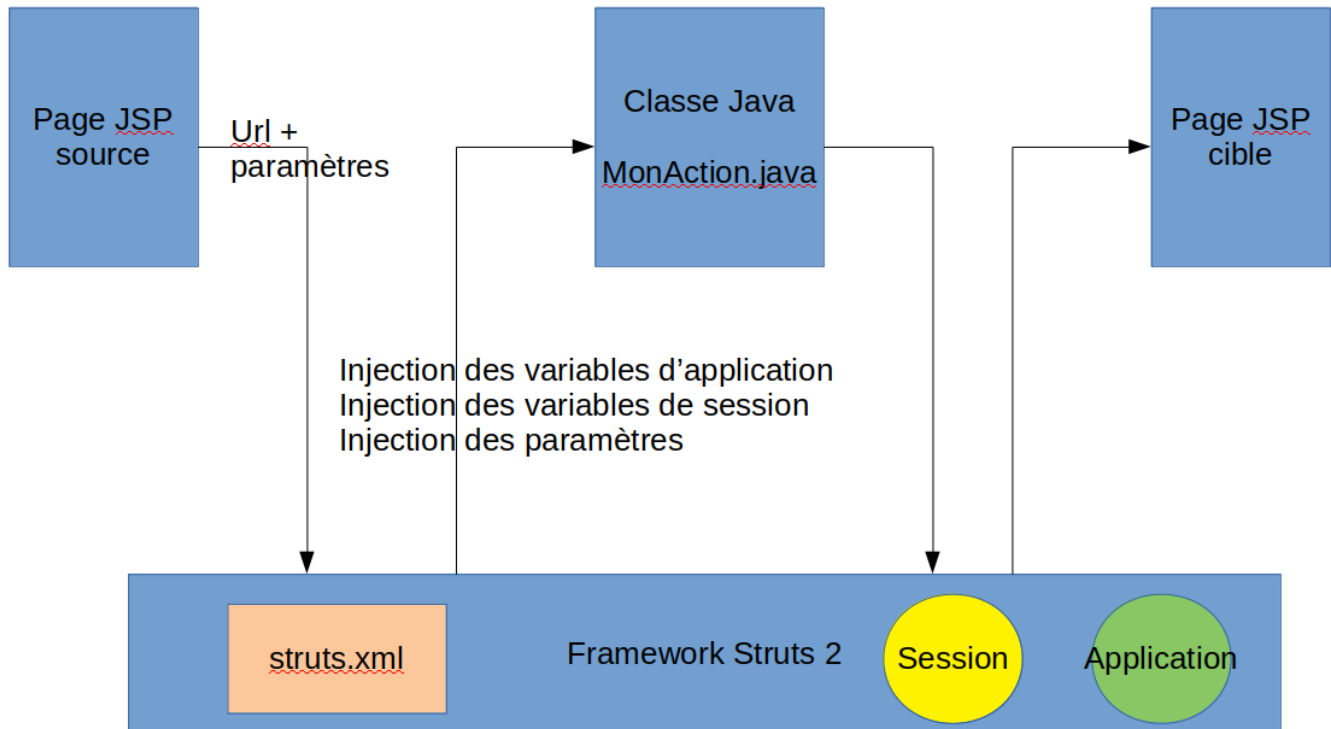
Matcher	Description
<code>eq(value)</code>	Vérifie que l'argument est exactement égal à value.
<code>isA(Class<T>)</code>	Vérifie que l'argument est une instance du type donné.
<code>same(object)</code>	Vérifie que l'argument est la même instance (comparaison par référence).
<code>isNull() / isNotNull()</code>	Vérifie si l'argument est null ou non.

7.24. Les ArgumentMatchers (3/3)

Matcher	Description / Exemple
<code>hasItems(...)</code>	Vérifie que la collection contient certains éléments. Exemple : <code>hasItems(hasProperty("name", is("Alice")), hasProperty("name", is("Bob")))</code>
<code>hasProperty("fieldName", matcher)</code>	Vérifie qu'un bean possède une propriété spécifique correspondant au matcher. Exemple : <code>hasProperty("id", is(42L))</code>
<code>is(...)</code>	Vérifie qu'un objet correspond exactement à la valeur attendue. Exemple : <code>is("Alice")</code>

Chapter 8. Tests structurels en Struts 2

8.1. Rappel de la machine Struts 2



8.2. Paris en ligne - action AjouterMatch (1/2)

L'action prend en entrée

- un sport (String)
- une équipe 1 (String)
- une équipe 2 (String)
- une date (String)

8.3. Paris en ligne - action AjouterMatch (2/2)

L'action a pour effet

- Ajouter un nouveau match dans le SI si les données sont renseignées et cohérentes
- Rendre le match construit accessible à la prochaine jsp
- Sinon dans tous les cas problématiques, un message d'erreur est préparé
 - Mauvais format de date
 - La date saisie est antérieure à la date courante
 - Une des informations attendues est vide ou non renseignée
 - l'équipe 1 et l'équipe 2 sont identiques

8.4. Action AjouterMatch (1/2)

```
@Override
public String execute() throws Exception {
    DateTimeFormatter formatter =
        DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm");
    LocalDateTime quandDF;
    try {
        quandDF = LocalDateTime.parse(quand, formatter);
    } catch (DateTimeParseException e) {
        addFieldError("quand", "mauvais format de date.");
        return INPUT;
    }
    if (quandDF.isBefore(LocalDateTime.now())) {
        addFieldError("quand",
            "la date doit être postérieure à aujourd'hui.");
        return INPUT;
    }
    long matchId = getFacade().ajouterMatch(getUtilisateur()
        .getLogin(), sport, equipe1, equipe2, quandDF);
    match = getFacade().getMatch(matchId);
    return SUCCESS;
}
```

8.5. Action AjouterMatch (2/2)

```
@Override
public void validate() {
    if (sport==null || sport.equals("")) {
        addFieldError("sport", "ne peut être vide.");
    }
    if (equipe1==null || equipe1.equals("")) {
        addFieldError("equipe1", "ne peut être vide.");
    }
    if (equipe2==null || equipe2.equals("")) {
        addFieldError("equipe2", "ne peut être vide.");
    }
    if (equipe1!=null && equipe2!=null && equipe1.equals(equipe2)) {
        addFieldError("equipe2", "ne peut être égal à équipe 1.");
    }
}
```

8.6. Coup d'oeil sur le struts.xml

```
<package name="/" extends="struts-default">
    ....
</package>
```

```

<action name="ajouterMatch" class="actions.AjouterMatch">
  <result name="success">
    /WEB-INF/vues/confirmerAjouterMatch.jsp
  </result>
  <result name="input">
    /WEB-INF/vues/ajouterMatch.jsp
  </result>
</action>
...
</package>

```

8.7. Comment tester une action ?

```

@Test
public void testAction() {
    AjouterPari ajouterPari = new AjouterPari();
    ...
}

```

- NON SURTOUT PAS !!!!
- LE CONTEXTE D'EXÉCUTION EST COMPLÈTEMENT IGNORÉ

8.8. Road to test AjouterPari action

Ajouter la librairie permettant de tester proprement une action

```

<dependency>
  <groupId>org.apache.struts</groupId>
  <artifactId>struts2-junit-plugin</artifactId>
  <version>2.5.22</version>
</dependency>

```

8.9. Road to test AjouterPari action

Ajouter la librairie permettant de tester proprement une action

```

public class TestAjouterMatch extends StrutsJUnit4TestCase {
    private static String etendreSiNecessaire(int x) {
        return x<10?"0"+x:Integer.toString(x);
    }
    @Test
    public void testOk() throws Exception {
        ...
    }
    @Test

```

```

    public void testK0DateAnterieure() throws Exception {
        ...
    }
    ...
}

```

8.10. Etapes pour un cas de test

- **Préparer les données** pour les **entrées** attendues
- **Préparer les données** attendues en **session**, en **application**
- **Injecter les maps** de session et d'application
- **Récupérer un objet ActionProxy** avec l'**uri** de l'action issue du **struts.xml**
- **Appeler** la méthode **execute()** sur l'**ActionProxy**
- **Vérifier que la clé retournée** par la méthode **execute()** est bien celle **attendue**
- **Vérifier** les éventuels **messages d'erreurs** injectés en récupérant l'**action** exécutée
- **Vérifier** que les éventuelles **données** attendues en **sortie** sont bien présentes

8.11. Exemple sur testOk()

Préparation des données

```

LocalDateTime now = LocalDateTime.now();
String annee = etendreSiNecessaire(now.getYear()+1);
String mois = etendreSiNecessaire(now.getMonthValue()+1);
String jour = etendreSiNecessaire(now.getDayOfMonth());
String heures = etendreSiNecessaire(now.getHour());
String minutes = etendreSiNecessaire(now.getMinute());
String notreDate = annee+"-"+mois+"-"+jour+" "+heures+":"+minutes;
DateTimeFormatter formatter =
    DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm");
LocalDateTime quand = LocalDateTime.parse(notreDate, formatter);

String football = "football";
String psg = "PSG";
String om = "OM";

```

8.12. Exemple sur testOk()

Injection en tant que paramètres

```

request.addParameter("sport", football);
request.addParameter("equipe1", psg);
request.addParameter("equipe2", om);

```

```
request.addParameter("quand",notreDate);
```

8.13. Exemple sur testOk()

Injection des maps

```
String pseudo = "Yohan";
long idMatch = 1;
Map<String,Object> sessions = new HashMap<>();
Map<String,Object> applications = new HashMap<>();

ActionProxy actionProxy = getActionProxy("/ajouterMatch");
actionProxy.getInvocation().getInvocationContext().setApplication(applications);
actionProxy.getInvocation().getInvocationContext().setSession(sessions);
```

8.14. Exemple sur testOk()

Partie qui pique (avec Mocks) et ajout des variables de session et d'application

```
// Définitions des mocks utilisateur et facadeParis en cohérence
// avec les appels prévus

sessions.put("user",utilisateur);
applications.put("facade",facadeParis);
```

8.15. Exemple sur testOk()

Exécution et assertions

```
String resultat = actionProxy.execute();
AjouterMatch actionExecutee = (AjouterMatch) actionProxy.getAction();

Assert.assertEquals(resultat,"success");
Assert.assertTrue(actionExecutee.getMatch()==match);
```

Récupération des messages d'erreurs éventuels

```
Collection<String> messages = actionExecutee
    .getFieldErrors().get("quand");
```

8.16. Tuto YouTube

Vous pouvez retrouver l'exemple ici : <https://youtu.be/ns1WX1Ifzco>

Chapter 9. Comment tester une API REST ?

9.1. Concept

- Comme en Struts, il va falloir isoler le contrôleur du reste
- Mocker tout ce qui n'est pas le contrôleur
- Simuler les requêtes et observer les réponses reçues

9.2. Des éléments de réponse

Technologies utilisées :

- Mockito : librairie populaire de mocks
- JUnit 5 : framework de test
- MockMvc : Permet de créer les requêtes et de tester la réponse
- @MockitoBean : Permet de mocker et de remplacer un bean dans SpringBoot
- Aucun besoin de dépendances supplémentaires à celles générées avec **SpringInitializer** pour une application classique REST

9.3. Exemple de service REST

```
public class Controleur {
    @Autowired
    FacadePromotion facadePromotion;
    //...
    @GetMapping("/etudiant/{numeroEtudiant}")
    public ResponseEntity<Etudiant> getEtudiant(@PathVariable String numeroEtudiant) {
        try {
            Etudiant etudiant = facadePromotion.getEtudiantById(numeroEtudiant);
            return ResponseEntity.ok(etudiant);
        } catch (EtudiantInexistentException e) {
            return ResponseEntity.notFound().build();
        }
    }
    //...
}
```

9.4. Méthodologie

- Identification
- Préparation
- Execution

9.5. Identification

- Identifier le nombre de cas à tester
 - Récupération OK
 - Récupération KO
- Identifier les codes retours
 - Récupération OK : **200**
 - Récupération KO : **404**
- Identifier les appels "extérieurs"
 - Ici : appel à la façade
- Identifier les différents paramètres : **@PathVariable**, **@RequestParam**, etc

9.6. Préparation

```
public ResponseEntity<Etudiant> getEtudiant(@PathVariable String numeroEtudiant) {  
    try {  
        Etudiant etudiant = facadePromotion.getEtudiantById(numeroEtudiant);  
        return ResponseEntity.ok(etudiant);  
    } catch (EtudiantInexistentException e) {  
        return ResponseEntity.notFound().build();  
    }  
}
```

- Préparer les appels des mocks : ici il y a juste l'appel à `getEtudiantById` à prévoir

9.7. Exécution

- Conception de la requête
 - le verbe
 - l'URI à attaquer
 - les différents paramètres à prévoir
- Définition des attentes sur la réponse
 - code retour attendu
 - Autres informations à observer : header, content, etc

9.8. Mise en application - Classe de test

```
@SpringBootTest  
@AutoConfigureMockMvc  
class EtudiantswsApplicationTests {
```

```

@Autowired
private MockMvc mvc;
@MockitoBean
FacadePromotion facadePromotion;

//TODO
}

```

9.9. Mise en application

```

@Test
public void testGetEtudiantByIdOK() throws Exception {
    // Objet retourné
    Etudiant etudiantRetourne = Mockito.mock(Etudiant.class);
    doReturn("Boichut").when(etudiantRetourne).getNomEtudiant();
    doReturn("Yohan").when(etudiantRetourne).getPrenomEtudiant();
    doReturn("Orleans").when(etudiantRetourne).getAdresse();
    doReturn("plus etudiant depuis longtemps")
        .when(etudiantRetourne).getNumeroEtudiant();
    // La façade retourne l'objet quand l'appel a lieu
    doReturn(etudiantRetourne).when(facadePromotion).getEtudiantById("1");
    // Requête
    mvc.perform(get("/mpws/etudiant/1").accept(MediaType.APPLICATION_JSON))
        .andExpect(status().isOk())
        .andExpect(result -> {
            System.out.println(result.getResponse().getContentAsString());
        });
}

```

9.10. Mise en application

```

@Test
public void testGetEtudiantByIdKO() throws Exception {
    doThrow(EtudiantInexistentException.class).when(facadePromotion)
        .getEtudiantById("1");
    mvc.perform(get("/mpws/etudiant/1").accept(MediaType.APPLICATION_JSON))
        .andExpect(status().isNotFound())
        ;
}

```

9.11. Quelques informations

- @MockBean, @SpyBean est une annotation dépréciée depuis Spring Boot 3.4+
- Elles sont remplacées par @MockitoBean et @MockitoSpyBean

Chapter 10. Tester une API REST SpringBoot 3 : JWT

10.1. Contexte

- Application Springboot 3 avec Spring Sec 6.0.2
- Authentification token JWT

10.2. Etude de cas

- Petit exemple disponible sur github : <https://github.com/yohanboichut/jwtspringboot3>
- Problématiques
 - Comment tester le code d'un endpoint non sécurisé ?
 - Comment tester le code d'un endpoint sécurisé ?

10.3. Politique de sécurité de l'application

Dans `SecurityConfig.java`

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {
    http.csrf(CsrfConfigurer::disable)
        .authorizeHttpRequests(authorize -> authorize
            .requestMatchers("/api/register").permitAll()
            .requestMatchers("/api/login").permitAll()
            .requestMatchers("/api/admin").hasRole("ADMIN")
            .requestMatchers("/api/user").hasRole("USER")
            .anyRequest().authenticated()
        )
    //...
    return http.build();
}
```

- Test de `/api/register` (non sécurisé)
- Test de `/api/admin` (sécurisé)

10.4. `/api/register`

```
@RequestMapping("/api")
public class Controleur {
```

```

private static final String TOKEN_PREFIX="Bearer ";
Facade facade; // Autowired
PasswordEncoder passwordEncoder; // Autowired
Function<Personne,String> genereToken; // Autowired
//...
@PostMapping("/register")
public ResponseEntity<String> enregistrar(@RequestParam String email,
@RequestParam String nom, @RequestParam String prenom,
@RequestParam String password){
    try {
        facade.enregistrarPersonne(email, nom, prenom, passwordEncoder.encode
(password));
        Personne j = facade.getPersonneById(email).get();
        return ResponseEntity.status(201)
            .header(HttpHeaders.AUTHORIZATION, TOKEN_PREFIX+genereToken.apply(j))
            .build();
    }
    catch (EmailDejaPrisException e) {
        return ResponseEntity.status(409).build();
    }
}
}

```

10.5. Les champs de ma classe de test

```

@Autowired
MockMvc mvc; // client REST
@MockBean
Facade facade; // façade de l'application
// On mocke la couche de façade car on s'intéresse qu'au contrôleur
@MockBean
PasswordEncoder passwordEncoder; //encodeur de passwords stockés
// on mocke le passwordEncoder pour contrôler les passwords stockés
@Autowired
ObjectMapper objectMapper; // JSON <-> Java
@Autowired
Function<Personne,String> genereToken; // Fonction de génération de token
// On utilise la réelle fonction de création de Token

```

10.6. Mise en application : inscription

```

@Test
public void testEnregistrarPersonne1() throws Exception {

    String email = "yohan.boichut@univ-orleans.fr";
    String nom = "Boichut";
    String prenom = "Yohan";
}

```

```

String password = "babar";
String passwordEncode = "celestine";
Role[] roles = new Role[]{Role.USER};

// On customize l'encodage du password
doReturn(passwordEncode).when(passwordEncoder).encode(password);
// On prévoit l'appel de la fonction getPersonneById
Personne personne = new Personne(email,nom,prenom,passwordEncode,roles);
doReturn(Optional.of(personne)).when(facade).getPersonneById(email);
// On laisse à Mockito gérer l'appel de enregistrerPersonne sur la
// façade i.e. c'est une méthode

mvc.perform(post(URI.create("/api/register")).contentType(MediaType
.APPLICATION_FORM_URLENCODED).content("email="+email+"&nom="+nom+"&prenom="+prenom+"&
password="+password))
    .andExpect(status().isCreated());
// On a lancé la requête et observé le code de retour
}

```

10.7. Mise en application : inscription /2

```

@Test
public void testEnregistrerPersonne2() throws Exception {

    String email = "yohan.boichut@univ-orleans.fr";
    String nom = "Boichut";
    String prenom = "Yohan";
    String password = "babar";
    String passwordEncode = "celestine";

    doReturn(passwordEncode).when(passwordEncoder).encode(password);
    doThrow(EmailDejaPrisException.class)
        .when(facade)
        .enregistrerPersonne(email,nom,prenom,passwordEncode);

    mvc.perform(post(URI.create("/api/register"))
        .contentType(MediaType.APPLICATION_FORM_URLENCODED)
        .content("email="+email+"&nom="+nom+"&prenom="+prenom+"&password="+password))
        .andExpect(status().isConflict());
}

```

10.8. /api/admin avec rôle ADMIN

```

@GetMapping("/admin")
public ResponseEntity<String> yeah(Principal principal){

```

```
        return ResponseEntity.ok("Yeahhh ! salut l'admin "+ principal.getName());
    }
```

- Nécessité d'avoir un token ADMIN d'après SecurityConfig

10.9. Ecriture du test avec token

```
@Test
public void testYeah() throws Exception {

    // On a besoin de récupérer un token pour l'authentification
    // Notre utilisateur doit être ADMIN
    String email = "yohan.boichut@univ-orleans.fr";
    String nom = "Boichut";
    String prenom = "Yohan";
    String password = "babar";
    Role[] roles = new Role[]{Role.ADMIN};
    Personne personneEnvoyee = new Personne(email, nom, prenom, password, roles);

    // On utilise le générateur de token pour créer des tokens valides
    String myToken = "Bearer "+genereToken.apply(personne);

    // Test de la fonctionnalité /admin
    mvc.perform(get(URI.create("/api/admin")).header("Authorization",myToken))
        .andExpect(status().isOk());
}
```

10.10. Possibilité de le faire sans Token généré

- Source : <https://www.baeldung.com/spring-oauth-testing-access-control>
- Utilisation de **MockMVC** pour simuler une authentification token
- Le code ci-dessous permet d'avoir un utilisateur authentifié avec :
 - comme rôle ADMIN
 - comme sujet : yohan.boichut@univ-orleans.fr

```
@Test
public void testYeah() throws Exception {

    mvc.perform(get("/api/admin").with(jwt()
        .authorities(List.of(new SimpleGrantedAuthority("ROLE_ADMIN")))
        .jwt(jwt -> jwt.claim(StandardClaimNames.SUB,
            "yohan.boichut@univ-orleans.fr"))))
        .andExpect(status().isOk());
}
```

Chapter 11. Comment tester une application Spring MVC ?

11.1. Principes

- Les principes sont les mêmes que ceux considérés dans le test de web-services
- La technologie est la même :
 - Utilisation de *MockMVC* pour avoir le client permettant de requêter le contrôleur
 - Utilisation des annotations *@MockitoBean* et *@MockitoSpyBean* pour mocker des composants extérieurs au contrôleur (en général façades des modèles)
 - Expression de la requête et des vérifications avec le *MockMVC* et des *matchers*

11.2. Exemple de contrôleur Spring MVC — `EvenementContrôleur.java`

```
// Composant injecté à la construction
private final GestionEvenements gestion;
//...
@PostMapping("/login")
public String authentifier(
    @Valid @ModelAttribute AuthenticationDTO authenticationDTO,
    BindingResult bindingResult,
    HttpSession session,
    RedirectAttributes redirectAttributes
) {
    if (bindingResult.hasErrors()) {

        return "login";
    }
    try {
        LaisserPasser laisserPasser = gestion.authentifier(authenticationDTO);
        session.setAttribute("laisserPasser", laisserPasser);
        return "redirect:/mesevenements/home";
    } catch (UtilisateurInconnuException e) {
        redirectAttributes.addFlashAttribute("erreur", "Identifiant ou mot de
passe incorrect.");
        return "redirect:/mesevenements/loginform";
    }
}
//...
```

11.3. Test structurel

- Identification des cas de tests
- Préparation de la requête et des vérification + mocks éventuels
- Execution

11.4. Identification

- Identifier le nombre de cas à tester : 3 cas de tests dans l'exemple
 - le cas où il y a des erreurs en input (AuthenticationDTO)
 - le cas où l'authentification se passe bien avec stockage d'un laisser-passer en variable de session
 - le cas où l'utilisateur est inconnu

11.5. Structure de la classe de Test

```
// Test du contrôleur EvenementControleur.java
@WebMvcTest(EvenementControleur.class)
public class TestEvenementControleur {

    // Mock du bean concernant la façade modèle
    @MockitoBean
    private GestionEvenements service;

    // récupération d'un client http
    @Autowired
    private MockMvc mockMvc;

    // méthodes de test
}
```

11.6. Premier cas de test - erreurs en input

```
// Composant injecté à la construction
private final GestionEvenements gestion;
//...
@PostMapping("/login")
public String authentifier(
    @Valid @ModelAttribute AuthenticationDTO authenticationDTO,
    BindingResult bindingResult,
    HttpSession session,
    RedirectAttributes redirectAttributes
) {
    if (bindingResult.hasErrors()) {
```

```

        return "login";
    }
    //...
}
//...

```

11.7. Premier cas de test - erreurs en input

- Il suffit de paramétrer la requête de telle sorte à ce que la validation prévue *@Valid* soit mise à l'épreuve
- Dans le cas présent :

```

public class AuthentificationDTO {
    @NotNull(message="L'email ne peut pas être vide")
    @NotBlank(message="L'email ne peut pas être vide")
    private String email;
    @NotNull(message="Le mot de passe ne peut pas être vide")
    @NotBlank(message="Le mot de passe ne peut pas être vide")
    private String motDePasse;
    // getters/setters/equals
}

```

11.8. Définition du cas de test

```

@Test
public void testAuthentifierK0ErrorInputs()
    throws UtilisateurInconnuException, Exception {
    // Mise en place de la requête et des vérifications
    mockMvc.perform(post("/mesevenements/login")
        .param("motDePasse", "pwd"))
    // Vérification que la vue retournée est login
        .andExpect(view().name("login"));
}

```

- Noter qu'il manque le paramètre **email** provoquant ainsi une violation de l'annotation *@NotNull(message="L'email ne peut pas être vide")* sur le champ **email** dans **AuthentificationDTO**
- Le *BindingResult* contient alors des erreurs et nous redirige vers la vue **login**
- Requête en POST comme décrit dans le contrôleur sur */mesevenements/login*

11.9. Deuxième cas de test - Tout va bien

```

@PostMapping("/login")

```

```

public String authentifier(
    @Valid @ModelAttribute AuthenticationDTO authenticationDTO,
    BindingResult bindingResult,
    HttpSession session,
    RedirectAttributes redirectAttributes,
    Model model
) {
    // pas d'erreur concernant l'email et le mot de passe récupérés dans
    authenticationDTO
    try {
        LaisserPasser laisserPasser = gestion.authentifier(authenticationDTO);
        session.setAttribute("laisserPasser", laisserPasser);
        return "redirect:/mesevenements/home";
    } catch (UtilisateurInconnuException e) {
        redirectAttributes.addFlashAttribute("erreur", "Identifiant ou mot de
        passe incorrect.");
        return "redirect:/mesevenements/loginform";
    }
}

```

11.10. Deuxième cas de test - Tout va bien

- Un appel à la façade modèle par *gestion.authentifier(authenticationDTO)*
- Récupération d'un laisser-passer suite à l'appel
- Stockage du laisser passer en variable de session

Il faut alors prévoir

- l'appel qui va être fait à la façade ainsi que son retour
- le stockage en variable de session

11.11. Définition du cas de test

```

@Test
public void testAuthentifierOK()
    throws UtilisateurInconnuException, Exception {
    // Ce qu'on espère récupérer côté contrôleur
    AuthenticationDTO authenticationDTO = new AuthenticationDTO("a@a.fr", "pwd");
    // le laisser-passer qui sera retourné par la façade gestion
    LaisserPasser mock = Mockito.mock(LaisserPasser.class);
    // si on appelle la méthode authentifier du bean gestion avec des paramètres qui
    // matchent avec l'objet prévu (authenticationDTO) alors on retourne le mock au
    // moment de cet appel
    given(gestion.authentifier(ArgumentMatchers.eq(authenticationDTO))).willReturn
    (mock);
    // Mise en place de la requête et des vérifications
    mockMvc.perform(post("/mesevenements/login").param("email", "a@a.fr").param(

```

```

    "motDePasse", "pwd"))
    // Vérification de la vue retournée
    .andExpect(view().name("redirect:/mesevenements/home"))
    // Vérification de la présence de la variable de session
    .andExpect(request().sessionAttribute("laisserPasser", is(mock)));
}

```

11.12. Troisième cas de test — UtilisateurInconnuException

- Un appel à la façade modèle par *gestion.authentifier(authenticationDTO)* levant une exception
- Il faut alors prévoir
- l'appel qui va être fait à la façade ainsi que sa levée d'exception

11.13. Définition du cas de test

```

@Test
public void testAuthentifierKOUtilisateurInconnu()
    throws UtilisateurInconnuException, Exception {
    // Ce qu'on espère récupérer côté contrôleur
    AuthenticationDTO authenticationDTO = new AuthenticationDTO("a@a.fr", "pwd");

    // si on appelle la méthode authentifier du bean gestion avec des paramètres qui
    // matchent avec l'objet prévu (authenticationDTO) alors on lève l'exception
    given(service.authentifier(ArgumentMatchers.eq(authenticationDTO))).willThrow
(UtilisateurInconnuException.class);
    // Mise en place de la requête et des vérifications
    mockMvc.perform(post("/mesevenements/login").param("email", "a@a.fr").param(
"motDePasse", "pwd"))
        // Vérification de la vue retournée
        .andExpect(view().name("redirect:/mesevenements/loginform"));
}

```

11.14. Outils indispensables

- Les fonctions statiques **post**, **get** qu'on retrouve dans *MockMvcRequestBuilders*
 - Utilisation de **param(nom,valeur)** pour ajouter des paramètres à la requête
 - Utilisation de **sessionAttrs** pour ajouter des variables de session à la requête
- les fonctions statiques **view**, **request** qu'on retrouve dans *MockMvcResultMatchers*
- les **argumentMatchers** de Mockito pour prévoir les appels aux mocks (voir section **Mockito**)